QUESTION 87

Title

Vertex Cover Approximation with Edge Selection Strategy

Aim

To design an approximation algorithm to find a vertex cover of a graph by repeatedly selecting uncovered edges and adding both endpoints into the cover set.

Problem Statement

Given an un directed graph, find an approximate vertex cover set that covers all edges of the graph.

The algorithm selects an uncovered edge, adds both vertices of that edge into the vertex cover, and removes all covered edges. This process continues until all edges are covered.

Input: Graph with vertices and edges.
Output: Approximate vertex cover set.

Introduction

A vertex cover is a set of vertices in a graph such that every edge has at least one endpoint in this set.

Finding the minimum vertex cover is an NP-Hard problem. Approximation algorithms are used to find a valid solution quickly for large graphs.

In this approach, an uncovered edge is selected using a greedy strategy. Both endpoints of the selected edge are added to the cover. The connected edges are removed, and the process continues until no edges remain.

Objectives

1. Find a valid vertex cover for a graph.
2. Select uncovered edges efficiently.
3. Apply approximation strategy.
4. Cover all graph edges.
5. Implement greedy edge selection.
6. Analyze complexity.

Constraints

- Every edge must be covered.
- The output must be a valid vertex cover.

- All edges should be processed.
- The algorithm should avoid unnecessary operations.

Algorithm

Step 1:
Input graph edges.

Step 2:
Create an empty vertex cover set.

Step 3:
While uncovered edges exist:

- Select an uncovered edge (u, v).
- Add u and v to the cover set.
- Remove all edges connected to u or v.

Step 4:
Repeat until all edges are covered.

Step 5:
Return the approximate vertex cover.

Pseudocode

```
Algorithm VertexCover(G)

Input:
   Graph G(V,E)

Output:
   Vertex Cover Set C

Begin

C = empty set

While E is not empty do

   Select an uncovered edge (u,v)

   Add u and v to C

   Remove all edges containing u or v

End While

Return C
```

End

Source Code

```
# Vertex Cover Approximation

edges = [(1,2), (2,3), (3,4), (4,1)]

cover = set()

while edges:

    u, v = edges[0]

    cover.add(u)
    cover.add(v)

    edges = [e for e in edges
            if u not in e and v not in e]

print("Approximate Vertex Cover:")
print(cover)
```

Sample Output

```
Approximate Vertex Cover:
{1, 2, 3, 4}
```

Working Example

Graph:

```
1 -------- 2
|          |
|          |
4 -------- 3
```

Edges:

(1,2), (2,3), (3,4), (4,1)

First select edge (1,2):

Cover = {1,2}

Next select edge (3,4):

Cover = {1,2,3,4}

All edges are covered successfully.

Complexity Analysis

Time Complexity

O(E)

Each edge is checked and removed during the algorithm.

Space Complexity

O(V)

Space is required to store the vertex cover set.

Applications:

- Computer networks.
- Resource management.
- Security systems.
- Scheduling problems.
- Graph optimization.
- Communication networks.

Result

The Vertex Cover Approximation Algorithm was successfully implemented using an edge selection strategy. The algorithm selected uncovered edges, added their endpoints to the cover set, and produced a valid approximate vertex cover covering all graph edges.